

Module 6: SQL Practice Queries

Complete concept-by-concept query reference — table used throughout: applicants (applicant_id, name, email, income, credit_score, application_date)

Setup: Database & Table Creation

```
-- Create the database
CREATE DATABASE loan_db;

-- Switch to the database
USE loan_db;

-- Create the applicants table with data types and constraints
CREATE TABLE applicants (
    applicant_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    income DECIMAL(10,2) CHECK (income >= 0),
    credit_score INT DEFAULT 0,
    application_date DATE NOT NULL
);
```

Concept: Data Types

```
-- Create a table demonstrating numeric, text, date, and boolean types
CREATE TABLE applicants_detailed (
    applicant_id INT PRIMARY KEY,
    name VARCHAR(50),
    income DECIMAL(10,2),
    application_date DATE,
    is_approved BOOLEAN
);
```

Concept: Constraints

```
-- Insert a row that violates the CHECK constraint (should fail)
INSERT INTO applicants (applicant_id, name, income, application_date)
VALUES (1, 'Test User', -500, '2026-08-13');

-- View table structure and constraints
DESCRIBE applicants;
```

Concept: Primary Key & Foreign Key

```
-- Create a related table with a foreign key referencing applicants
CREATE TABLE loan_decisions (
    decision_id INT PRIMARY KEY,
    applicant_id INT,
    decision VARCHAR(10),
    FOREIGN KEY (applicant_id) REFERENCES applicants(applicant_id)
);
```

```
);
```

```
-- Insert a decision linked to an existing applicant
```

```
INSERT INTO loan_decisions VALUES (1, 2, 'Approved');
```

Concept: INSERT

```
-- Insert a single row
```

```
INSERT INTO applicants (applicant_id, name, email, income, credit_score, application_date)
VALUES (2, 'Asha Menon', 'asha@mail.com', 45000.00, 720, '2026-08-13');
```

```
-- Insert multiple rows in one statement
```

```
INSERT INTO applicants (applicant_id, name, email, income, credit_score, application_date)
VALUES
    (3, 'Ravi Kumar', 'ravi@mail.com', 31000.00, 590, '2026-08-10'),
    (4, 'Priya Nair', 'priya@mail.com', 68000.00, 745, '2026-08-11');
```

```
-- Insert filtered rows copied from another query (INSERT INTO ... SELECT)
```

```
INSERT INTO high_income_applicants (applicant_id, name, income)
SELECT applicant_id, name, income
FROM applicants
WHERE income > 50000;
```

Concept: SELECT

```
-- Select all columns and rows
```

```
SELECT * FROM applicants;
```

```
-- Select specific columns only
```

```
SELECT name, income FROM applicants;
```

Concept: UPDATE

```
-- Update a specific row using WHERE
```

```
UPDATE applicants
SET credit_score = 680
WHERE applicant_id = 3;
```

```
-- Update multiple columns at once
```

```
UPDATE applicants
SET income = 35000.00, credit_score = 610
WHERE applicant_id = 3;
```

Concept: DELETE

```
-- Delete a specific row using WHERE
```

```
DELETE FROM applicants WHERE applicant_id = 4;
```

```
-- Delete all rows but keep the table structure
```

```
DELETE FROM applicants;
```

Concept: WHERE Clause (Filtering)

-- Filter using a comparison operator

```
SELECT * FROM applicants WHERE credit_score > 600;
```

-- Filter using AND / OR / range / pattern / list

```
SELECT * FROM applicants WHERE credit_score > 600 AND income > 40000;
```

```
SELECT * FROM applicants WHERE income BETWEEN 30000 AND 60000;
```

```
SELECT * FROM applicants WHERE name LIKE 'A%';
```

```
SELECT * FROM applicants WHERE applicant_id IN (2, 3, 5);
```

Concept: ORDER BY (Sorting)

-- Sort by one column, descending

```
SELECT * FROM applicants ORDER BY income DESC;
```

-- Sort by multiple columns (tiebreaker)

```
SELECT * FROM applicants ORDER BY credit_score DESC, income DESC;
```

Concept: LIMIT

-- Get the top 3 rows by income

```
SELECT * FROM applicants ORDER BY income DESC LIMIT 3;
```

-- Paginate results using LIMIT with OFFSET

```
SELECT * FROM applicants ORDER BY applicant_id LIMIT 2 OFFSET 2;
```

Concept: DISTINCT

-- Get unique credit_score values

```
SELECT DISTINCT credit_score FROM applicants;
```

-- Get unique combinations across two columns

```
SELECT DISTINCT credit_score, application_date FROM applicants;
```

Concept: Combined Filtering & Sorting

-- Combine WHERE, DISTINCT, ORDER BY, and LIMIT in one query

```
SELECT DISTINCT credit_score  
FROM applicants  
WHERE income > 40000  
ORDER BY credit_score DESC  
LIMIT 5;
```

Concept: COUNT

-- Count all rows

```
SELECT COUNT(*) FROM applicants;
```

-- Count only non-NULL values in a column

```
SELECT COUNT(credit_score) FROM applicants;
```

Concept: SUM

-- Total income across all applicants

```
SELECT SUM(income) FROM applicants;
```

-- Total income only for applicants with credit_score above 600

```
SELECT SUM(income) FROM applicants WHERE credit_score > 600;
```

Concept: AVG

-- Average income across all applicants

```
SELECT AVG(income) FROM applicants;
```

-- Average credit_score for applicants earning above 40000

```
SELECT AVG(credit_score) FROM applicants WHERE income > 40000;
```

Concept: MIN & MAX

-- Find the lowest credit_score

```
SELECT MIN(credit_score) FROM applicants;
```

-- Find the lowest, highest, and average income in one query

```
SELECT MIN(income), MAX(income), AVG(income) FROM applicants;
```

Concept: GROUP BY (with Aggregate Functions)

-- Count applicants per credit_score value

```
SELECT credit_score, COUNT(*)  
FROM applicants  
GROUP BY credit_score;
```

-- Average income per credit_score group

```
SELECT credit_score, AVG(income)  
FROM applicants  
GROUP BY credit_score;
```

Concept: HAVING (Filtering Groups)

-- Find duplicate emails (groups with more than 1 row)

```
SELECT email, COUNT(*) AS occurrences  
FROM applicants  
GROUP BY email  
HAVING COUNT(*) > 1;
```

-- Find duplicate full rows across multiple columns

```
SELECT name, income, credit_score, COUNT(*) AS occurrences  
FROM applicants  
GROUP BY name, income, credit_score  
HAVING COUNT(*) > 1;
```

Concept: Data Consistency Checks

```
-- Attempt an insert that breaks referential integrity (should fail)
INSERT INTO loan_decisions VALUES (2, 999, 'Approved');

-- Attempt an insert that breaks a domain/CHECK rule (should fail)
INSERT INTO applicants (applicant_id, name, income, application_date)
VALUES (10, 'Test Consistency', -1000, '2026-08-14');
```

Concept: Data Quality Checks

```
-- Check missing value rate for a column
SELECT
    COUNT(*) AS total_rows,
    COUNT(credit_score) AS non_null_scores,
    COUNT(*) - COUNT(credit_score) AS missing_scores
FROM applicants;

-- Check distribution skew across a column
SELECT credit_score, COUNT(*)
FROM applicants
GROUP BY credit_score
ORDER BY COUNT(*) DESC;
```

Concept: Practical / Scheduled Data Retrieval

```
-- Retrieve only yesterday's new applicants
SELECT * FROM applicants WHERE application_date = CURDATE() - 1;

-- Retrieve only complete, valid rows for training
SELECT * FROM applicants
WHERE credit_score IS NOT NULL AND income IS NOT NULL;
```

Summary: Concepts Covered

1. Database & table creation (CREATE DATABASE, CREATE TABLE)
2. Data types (numeric, text, date, boolean)
3. Constraints (NOT NULL, UNIQUE, CHECK, DEFAULT)
4. Primary key & foreign key relationships
5. INSERT (single row, multi-row, INSERT INTO ... SELECT)
6. SELECT (all columns, specific columns)
7. UPDATE (single column, multiple columns)
8. DELETE (targeted row, all rows)
9. WHERE clause (comparison, AND/OR, BETWEEN, LIKE, IN)
10. ORDER BY (single column, multi-column sort)
11. LIMIT (top-N, pagination with OFFSET)
12. DISTINCT (single column, multi-column)
13. Combined filtering & sorting (WHERE + DISTINCT + ORDER BY + LIMIT)
14. COUNT()

15. SUM()
16. AVG()
17. MIN() & MAX()
18. GROUP BY with aggregate functions
19. HAVING (filtering groups, finding duplicates)
20. Data consistency checks (referential integrity, domain integrity)
21. Data quality checks (missing values, distribution skew)
22. Practical / scheduled data retrieval patterns